Syscall Shellcode in WoW64 Windows

Bramwell Brizendine[1] and Tarek Abdelmotaleb[2]

[1] University of Alabama in Huntsville, Huntsville, AL, USA
[2] Dakota State University, Madison, SD, USA
bramwell.brizendine@uah.edu

Abstract. Syscalls in Windows have become very trendy, allowing red team software to use direct syscalls to evade EDR. Windows syscalls have virtually never been done in shellcode, except for the special case of Egghunter shellcode, which typically uses one syscall, as part of the process to search through process memory for a secondary shellcode. The reason why syscalls have been largely avoided in shellcode is because of the portability problem with syscalls, as Windows releases change the syscall values (SSNs) provided to the eax register to initiate the shellcode. Windows 10, for instance, has more than thirteen OS releases. This research provides a methodology to overcome the portability problem associated with shellcode. It also provides a methodology for how to utilize syscalls in shellcode. Finally, this research presents a new tool, ShellWasp, which builds templates of syscall shellcode. ShellWasp utilizes a novel syscall array, which can be populated at runtime with the correct SSNs, allowing syscall shellcode to be able to function across multiple OSs and OS releases. ShellWasp builds the shellcode's underlying infrastructure, so that it can effectively use the syscall array at runtime, without the need for using hardcoded syscall values. ShellWasp additionally labels all syscall parameters, making it easier for shellcode authors to understand how to construct syscall shellcode.

Keywords: Shellcode, Windows syscalls, Reverse Engineering, Offensive Security

1 Introduction

Windows syscalls in shellcode have only been used on extremely rare occasions, with the single exception of being used in the context of Egghunters. In fact, outside of Egghunters, we know of only one instance of a non-Egghunter example of syscalls being used in shellcode, and that is from 2005, the Windows XP era. The challenge with syscalls always is the fact that the syscall service numbers (SSNs) change with each release of Windows, creating a problem of portability, where syscall values work in one OS release, but are entirely different in another. That was not as much of a problem in the Windows XP era, as the changes corresponded to new service packs, but it has been exacerbated in the modern era, where Windows 10 has more than thirteen releases, as well as some others we are not including in our count. The fact that the SSNs change so frequently can make them unreliable to use, as it could be guesswork as to which SSN would be required on a target's OS. One wrong choice, and the entire shellcode does not work.

In this paper, we present a method to overcome the problem of portability, and we provide a tool to automatically construct a template for shellcode with syscalls on the 32-bit, WoW64 platform, with the tool being able to identify the correct release and save the appropriate SSNs into a syscall array, allowing elements of the array to be supplied at runtime when the syscall is made – rather than relying upon hardcoded values. Additionally, one of the greatest challenges with shellcode utilizing syscalls is that they can be considerably more difficult to do than with shellcode utilizing Windows APIs, and a large part of that is the lack of previous work in this area. Thus, in this paper we also provide a methodology for how to construct syscall shellcode, something that has been lacking until this research.

This paper is organized as follows. Firstly, we provide a detailed background on syscalls in Windows and in particular in a 32-bit, WoW64 environment. This stems from original research we conducted utilizing reverse engineering tools as well as the existing literature. This background also covers syscall usage in general, although all other syscall usage is outside the realm of shellcode. Next, we propose a solution for the portability problem of syscalls in WoW64 shellcode. Third, we discuss, ShellWasp, which generates a template for shellcode in WoW64. Fourth, we provide a methodology for creating shellcode with syscalls. Finally, we provide validation by providing a case study of the methodology.

2 Background

While seldom discussed in academia, Windows syscalls have become popular recently [1], although they have been used for more than 15 years for offensive purposes, such as Egghunters [2, 3]. An egghunter is a specialized shellcode designed to find a larger shellcode or egg injected into process memory at an unknown location. To find the egg, a syscall such as NtAccessCheckAndAuditAlarm is used to check if memory is valid, before checking that memory location for a distinctive tag, e.g. w00tw00t. Once the tag is found, control flow can transfer to the newly found egg.

NTDLL functions, when reverse engineered, can indicate the System Service Number (SSN) of the syscall, and this information has been aggregated and shared by the security community [4]. In 32-bit applications of Windows, a syscall can be made by calling a function whose address is pointed to by offset 0xC0 of the TIB, with the TIB being pointed to by the fs register. Thus, malicious authors can invoke a syscall by loading the SSN into eax and calling the function pointed to by that offset.

In testing Windows syscalls, we created a number of these painstakingly, both for Windows 7 and Windows 10, reverse engineering them from normal usage in Windows applications. In general, Windows syscalls require significantly more complex setup than WinAPI functions.

In this section, we provide a helpful background to better understand Windows syscalls in shellcode. Firstly, we provide a historical background on syscalls in historical versions of Windows, which we largely are not concerned with. We focus heavily on Windows 7, 10, and 11. We intentionally omit Windows 8. Most of the sections on Windows 7, 10, and 11 derive from original research efforts made through reverse engineering, although other sources are consulted as well to best understand what was discovered. We then survey the current and leading techniques for syscall usage, albeit outside the context of shellcode. Finally, we discuss intercepting syscalls at kernel-mode, highlighting one of the great values for using syscalls in the first place.

2.1 Historical Implementations of Syscalls in Windows

Syscalls have been regularly used as part of Egghunters, and literature [2] exists that discuss that usage. It is intended simply to identify if memory is valid and is safe to check to see if a special tag is present, to avoid causing an access violation by checking memory that was not committed. Egghunters have existed since at least 2006, and likely several years longer than that.

In 2005, Piotr Bania created a shellcode [5] utilizing four syscalls, to cause a binary to be executed upon restarting a computer, while also providing a method of persistence for this binary. With this shellcode, Bania also provides a brief paper describing his shellcode and his thoughts on creating syscalls in shellcode. To our knowledge, this is the only non-Egghunter example of a shellcode with syscalls that has been disseminated to the public. While his work is high quality, Windows internals have changed, and his shellcode is no longer effective, and the method of calling syscalls additionally has changed. The syscall portability problem is not addressed in his work.

Outside of Egghunters, syscalls largely were not used in Windows, although occasionally in malware. In 2018, prior to modern red team, syscall tools, there was a report [6] of a sharp increase in syscall usage in some malware, which involved usage of dual loading of NTDLL. Gavriel provided hashes for eight malware using syscalls, including instances of LockPos, Flokibot, Trickbot, Formbook, Osiris, Neurevt, Fastcash, and Coininer. Gavriel's report was directly responsible for the proliferation of syscall tools that would follow.

2.2 Syscalls in Windows 7 WoW64

In Windows 7, WoW64 provides a new implementation of syscalls. When looking at the NTDLL functions that correspond to syscalls, we see that the Thread Information Block (TIB) is being used, as the TIB is always pointed to by the fs register. At offset 0xC0, it points to a function called X86SwitchTo64BitMode in wow64cpu.dll, as can be seen in the below figure. The SSN, as always, is loaded into eax. Stack cleanup occurs after the call to fs:[0xc0]; four bytes is added to esp to accommodate for the address of the next instruction added for the call. Next, additional stack cleanup occurs with the ret. The syscall use in Windows 7 WoW64 is to cleanup

the syscall parameters with this form of ret. In the example below, *ret 18h* is used. The 0x18 accounts for the six parameters for NtAllocateVirtualMemory; each takes up four bytes, so that is a total of 24, or 0x18, bytes.

```
0:009> u ntdll!ntallocatevirtualmemory
ntdll!NtAllocateVirtualMemory
777ffac0 b815000000      mov     eax,15h
777ffac5 33c9            xor     ecx,ecx
777ffac7 8d542404        lea     edx,[esp+4]
777ffacb 64ff15c0000000  call    dword ptr fs:[0C0h]
777ffad2 83c404          add     esp,4
777ffad5 c21800          ret     18h
```

Fig. 1. An edited snippet from WinDbg shows fs:[0xC0] being used to facilitate syscalls.

This function is used to make a far jump that goes to to CpupReturnFromSimulatedCode in wow64cpu.dll, as seen in the figure below. WoW64cpu.dll is a 64-bit library loaded into 32-bit address space, and by default it is hidden from the PEB, so when that address space appears in a debugger, it is not labelled [7]. The far jump is used to transition from 32-bit to 64-bit mode; this where the syscall itself takes place. The 0x33 code segment selector denotes that it will be transitioning to 64-bit mode; the 0x23 selector would denote transitioning to 32-bit mode. WoW64 is simply providing an implementation to emulate 32-bit Windows on a 64-bit platform.

```
0:009> dd fs:c0
0053:000000c0  73962320 00000409 00000000 00000000

0:009> u 73962320
73962320 ea1e2796733300  jmp     0033:7396271E
73962327 0000            add     byte ptr [eax],al
```

This far jump lets us transition from 32-bit to 64-bit code.

Fig. 2. Fs:[0xC0] leads to a far jump, allowing a transition from 32-bit to 64-bit mode.

From the above figure, it is apparent that there will be a jump to 0x7396271E. However, if we go to that location in 32-bit mode, the code is interpreted incorrectly in the debugger, which is 32-bit. Because the code is being interpreted incorrectly, it is not accurate, as can be seen in the figure below. Because wow64cpu.dll is a 64-bit library in 32-bit address space, it provides the final code that is required to transition from user-mode to kernel-mode. The inner workings of such are beyond the scope of this paper and our research. From this point, we take a black box approach, where we simply know that each time a syscall is made in WoW64, there is a transition from 32-bit to 64-bit mode, going from user-mode to kernel-mode. We are not concerned with implementation details beyond this point. To follow along beyond this point, it would be necessary to do kernel-debugging, where an entire operating system is debugged by another.

32-bit

```
0:009> uf 7396271E
7396271e 6744            inc     esp
73962720 8b0424          mov     eax,dword ptr [esp]
73962723 45              inc     ebp
73962724 8985bc000000    mov     dword ptr [ebp+0BCh],eax
7396272a 41              inc     ecx
7396272b 89a5c8000000    mov     dword ptr [ebp+0C8h],esp
73962731 49              dec     ecx
73962732 8ba42480140000  mov     esp,dword ptr [esp+1480h]
```

64-bit

```
0:  67 44 8b 04 24          mov    r8d,DWORD PTR [esp]
5:  45 89 85 bc 00 00 00    mov    DWORD PTR [r13+0xbc],r8d
c:  41 89 a5 c8 00 00 00    mov    DWORD PTR [r13+0xc8],esp
13: 49 8b a4 24 80 14 00    mov    rsp,QWORD PTR [r12+0x1480]
```

Fig. 3. Following the far jump in 32-bit mode leads to the code being interpreted incorrectly by WinDbg. The actual bytes when interpreted as 64-bit code produce the correct disassembly.

Windows 7 WoW64 easily lends itself to syscall shellcode, as all that is required is to set up parameters for the syscalls, and then make a dereferenced call to fs:[0xc0], *call dword ptr fs[0xc0]*. It is the necessary to do stack cleanup after the syscall, removing the parameters as well as the address added by the call.

Windows 7 is known to only have two different releases with different syscall values, Service Pack 0 and Service Pack 1, with OS builds ending in 0x1DB0 and 0x1DB1.

2.3 Syscalls in Windows 10 WoW64

Syscalls in Windows 10 differ from those of Windows 7, as the method by which they are called in NTDLL has been changed.

```
0:000> u ntdll!ntallocatevirtualmemory
ntdll!NtAllocateVirtualMemory:
76fe2b10 b818000000      mov     eax,18h
76fe2b15 ba1088ff76      mov     edx,offset ntdll!Wow64SystemServiceCall (77358870)
76fe2b1a ffd2            call    edx
76fe2b1c c21800          ret     18h
76fe2b1f 90              nop

0:000> u 77358870
ntdll!Wow64SystemServiceCall:
77358870 ff2528923f77    jmp     dword ptr [ntdll!Wow64Transition (773f9228)]
```

18h = SSN for NtAllocateVirtualMemory

Fig. 4. Windows 10 WoW64 syscall for NtAllocateVirtualMemory.

As seen above, Windows 10 marks a significant departure from Windows 7 in terms of implementation of syscalls in WoW64. Now when we disassemble an NTDLL function, the fs:[0xc0] offset is not used to enable us to transition from 32-bit to 64-bit mode and perform the syscall. Instead, there is a function used from NTDLL, Wow64SystemServiceCall. This in turn then makes a dereferenced jump to ntdll!Wow64Transition, as can be seen above. Wow64SystemServiceCall then leads us to the necessary far jump in wow64cpu.dll, CpupReturnFromSimulatedCode, as can be seen in the figure below.

```
0:000> u 77358870
ntdll!Wow64SystemServiceCall:
77358870 ff2528923f77     jmp     dword ptr [ntdll!Wow64Transition (773f9228)]

0:000> dd 773f9228
76f67000  76f67000 77099000 00000000 00000000

0:000> u 76f67000
76f67000 ea0970f6763300   jmp     0033:76F67009
76f67007 0000             add     byte ptr [eax],al
76f67009 41               inc     ecx
76f6700a ffa7f8000000     jmp     dword ptr [edi+0F8h]
```

This far jump lets us transition from 32-bit to 64-bit code.

Fig. 5. In Windows 10 WoW64 the far jump to 64-bit mode can be reached from ntdll!Wow64Transition.

While the normal method of calling syscalls has changed in Windows 10 WoW64, for purposes of backward computability, the old Windows 7 way of invoking syscalls with fs:[0xc0] has been maintained. If we examine the fs:[0xc0] offset in WinDbg, we see that it points to the identical location as what was pointed to by ntdll!Wow64Transition, as shown in the figure below. The fact that fs:[0xc0] still points to the same location enables us to continue to use it for making shellcode with syscalls on Windows 10 WoW64. Likely, if this were not the case, ntdll!Wow64Transition could be found via PEB walking and searching through the exports directory, although we have not attempted this, although some mitigations could be problematic in some situations. Once found though, we would be able to reach the far jump.

```
0:000> dd fs:c0
0053:000000c0  76f67000  00000409 00000000 00000000
```

Fig. 6. Windows 10 WoW64 maintains allows the far jump to 64-bit mode to be reached still via fs:[0xc0].

In Windows 10, when we examine the far jump and where we lead us, we see that the location to CpupReturn-FromSimulatedCode can now only be reached via 64-bit code that points to a register that does not exist in 32-bit mode [8]. The r15+0xf8 maintains the address for this function. The 32-bit code show below, which may appear correct, is deceptive, as the far jump with the 0x33 selector will cause the code at 0x76f67009 to be executed as 64-bit. As before, CpupReturnFromSimulatedCode will establish the proper context and perform the actual system call.

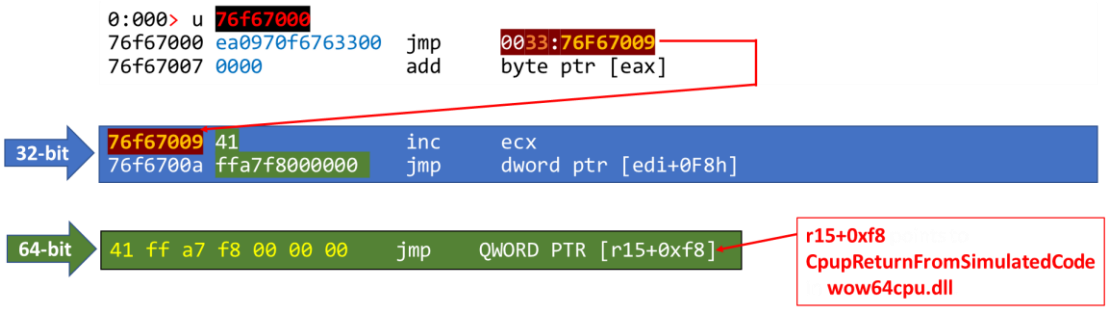

Fig. 7. In Windows 10 WoW64, the far jump leads us to CpupReturnFromSimulatedCode.

In examining the disassembly for syscalls made in Windows 10, we see that unlike with Windows 7, there is no *add esp, 0x04*. The stack parameters are cleaned up as before with a *ret*, and the number of bytes to be removed. For instance, a *ret 18* would remove 24 bytes (0x18 bytes) from the stack.

2.4 Syscalls in Windows 11 WoW64

In Windows 11 WoW64, the usage of syscalls at least from the perspective of what is seen from NTDLL is virtually identical to Windows 10 WoW64, although a different API is used initially. The function ntdll!RtlInterlockedCompareExchange64+0x180 is what is used to call ntdll!Wow64Transition, the same function called in Windows 10. Everything else is virtually identical, at least on the user-mode side, and for this research, looking at what is happening in kernel-mode is out of scope. Wow64Transition provides an address that, once dereferenced, is the far jump with the 0x33 selector, allowing us to transition from 32-bit to 64-bit mode.

```
0:000> u ntdll!ntallocatevirtualmemory
ntdll!NtAllocateVirtualMemory:
77884d50 b818000000      mov     eax,18h
77884d55 ba408f8a77      mov     edx,offset ntdll!RtlInterlockedCompareExchange64+0x180
(778a8f40)
77884d5a ffd2            call    edx
77884d5c c21800          ret     18h
77884d5f 90              nop
```

18h = SSN for NtAllocateVirtualMemory

```
0:000> u  778a8f40
ntdll!Wow64SystemServiceCall:
778a8f40 ff2520c29377    jmp     dword ptr [ntdll!Wow64Transition (7793c220)]
778a8f46 cc              int     3

0:000> dd 7793c220
7793c220  77806000 7793c000 00000000 00000000

0:000> u 77806000
77806000 ea096080773300  jmp     0033:77806009
```

Fig. 8. Windows 11 WoW64 is very similar to Windows 10 at the user-mode level.

As with Windows 10, the fs:[0xc0] still points to the same address as ntdll!Wow64Transition. Thus, once it is dereferenced, it will lead us to the same far jump to transition from x86 to x64 mode, as can be seen below. With respect to syscall usage in WoW64 shellcode, usage is identical to Windows 10 WoW64.

```
0:000> dd fs:c0
0053000000c0  77806000 00000409 00000000 00000000
```

Fig. 9. In Windows 11 WoW64, fs:[0xc0] points to the same far jump as does ntdll!Wow64

At the time of present writing, there is only one official release of Windows 11, which is 21H2, or OS build 20348. However, there have been several Insider Preview builds, which seem to come out monthly. We have not examined the preview builds to see how much they differ from 21H2, although we suspect they are likely incremental changes.

2.5 Changes in Syscall SSNs on Windows 10 and Windows 11

Windows syscalls present a lack of portability, making usage more challenging, as the numbers can change unpredictably. In looking at recent builds for Windows 10 and Windows 11, we had obtained some results from a tool that had parsed NTDLL. The tool looked for only one-byte, truncating part of the SSN. In creating the lists of syscalls for some recent builds, WinDbg was used to examine NTDLL, for all syscalls, checking for accuracy and making corrections. In so doing, dozens of inaccuracies were found, due to truncation. What was noteworthy

was that some syscalls were as long as six digits; instances were seen of SSNs being one, two, three, four, five or six digits long. Having been very familiar with the Windows Syscall Table created by Mateusz "j00ru" Jurczyk, we were very much aware that all instances we had seen for Windows 10 were between one and three digits. Jurczyk's syscall table has not been updated in nearly two years, so individuals who use tools to create alternative lists of syscalls are urged to be cautious, due to potential inaccuracies from said tools. It is possible this could be a technique from Microsoft to make some tools to parse NTDLL for syscalls create inaccurate results. It is also entirely possible this has occurred through pure happenstance. The best method to provide results with no errors is simply to utilize WinDbg, as shown below. While we manually did this, using a list of commands we generated programmatically, a simple script could be created to parse the results of WinDbg, greatly simplifying the labor and assuring that the results produced were error-free. While our manual effort of checking WinDbg results took only 45 minutes per OS build, by adding additional automation to parse the WinDbg results, this could very significantly reduce the time required to a few minutes.

```
0:000> u ntdll!NtWriteFile L1
ntdll!NtWriteFile:
77652a30 b808001a00      mov     eax,1A0008h
0:000> u ntdll!NtWriteFileGather L1
ntdll!NtWriteFileGather:
77652b80 b81b001a00      mov     eax,1A001Bh
0:000> u ntdll!NtWriteRequestData L1
ntdll!NtWriteRequestData:
77652f40 b857000000      mov     eax,57h
0:000> u ntdll!NtWriteVirtualMemory L1
ntdll!NtWriteVirtualMemory:
77652d70 b83a000000      mov     eax,3Ah
0:000> u ntdll!NtYieldExecution L1
```

Fig. 10. In Windows 10 build 21H2, we see very long SSNs, which are longer than those from earlier builds.

In the excellent Hell's Gate white paper [9], the author makes an argument that SysWhispers was problematic in that it was reliant upon the work for Mateusz Jurczyk, as it could not dynamically self-resolve syscalls. The implication is that if Jurczyk were to end his work, such tools would no longer be reliable, and thus SysWhispers or similar tools that require lists of syscall SSNs would not be visible. While that is true, and while it is also true that some existing tools to parse NTDLL to programmatically determine syscalls may be error-prone, due to longer SSNs, the above demonstrates that any user could generate a list of commands for WinDbg and parse the results, to discover an accurate listing of syscalls for a particular OS build. Thus, we would argue that while there are other methods, we need not necessarily be reliant upon one individual for that labor, as the work associated with generating those lists can now be greatly simplified.

2.6 Windows Syscalls Outside of Shellcode

A 2018 article by Gavriel describes a sharp increase in syscalls usage in malware. Many offensive security tools have emerged since that time, starting in 2019, with SysWhispers. Multiple techniques have been developed, each addressing the portability problem with syscalls, helping to address how the correct syscall SSNs can selected for the correct Windows release being used. All of the known projects have addressed this from a 64-bit standpoint, and to our knowledge they have all been used as a way to weaponize using syscalls in larger projects, such as C++, C#, etc., by providing Assembly stubs or headers that could be used, or by dynamically resolving the correct syscall values, either on the fly or to store them in memory for later use.

Dumpert. In June 2019, Dumpert [10, 11] was released as the first proof of concept tool using syscalls that was developed in response to Gabriel's report on syscall usage in malware. Dumpert uses precomputed syscalls list, and it uses the native API RtlGetVersion to determine OS version, saving that information, and allowing the

correct syscall SSNs to be used for syscalls, as determined by the OS release. Dumpert is able to build proof of concept code, utilizing syscalls, to perform LSASS memory dumps with Cobalt Strike.

SysWhispers. Released in December 2019, SysWhispers by Jackson T. was a Python tool that that generated header and ASM pairs for all syscalls in ntoskrnl.exe, for all versions of Windows from XP through Windows 10 20H2. This work was not intended for shellcode – as indeed, no syscall tools other than ShellWasp have been. This tool, as are all the syscall tools we are aware of, are all designed to work with the 64-bit implementation of syscalls. This work builds upon what was done in Dumpert. SysWhispers was the first tool to rely upon using the PEB to identify the OS build. As SysWhispers was 64-bit, it queried GS:[0x60] and then offset 0x120 to determine the OS Build. Prior to checking the OS Build, it checked the major version of Windows at offset 0x118 and the minor version of Windows at offset 0x11c. By utilizing the PEB, the exact syscalls could be determined, by using SSNs from the syscall table prepared by Jurczyk [4]. SysWhispers generally generates very long Assembly to determine specific OS major and minor versions as well as the OS build, and it produced an Assembly stub.

FreshyCalls. Released in December 2020, FreshyCalls [12, 13] utilizes a technique developed by its author, ElephantSe4l, to programmatically determine syscall values on the fly. It sorts addreseses beginning with Nt that are contained in the exports directory. Once addresses are sorted, it then increments each by 1, thereby determining the syscall value without needing to parse the stub.

SysWhispers2. SysWhisphers2 [14] is re-imagining of SysWhispers, to generate header and Assembly files for x64/AMD64 platforms. SysWhispers2 was released shortly after FreshyCalls, in January 2021. SysWhispers2 utilizes ElephantSe4l's sorting by address technique, with a slight variation. As with FreshyCalls, the Export Address Table of NTDLL is parsed and addresses are sorted. The primary difference is that now it begins with NTDLL functions beginning with Zw, instead of Nt. These are then sorted to be able to programmatically determine the necessary SSNs without need to use precomputed SSNs, such Jurczyk's syscall table [4]. SysWhisphers2 allows the user to utilize an implant of the syscall stub, which is often just a few lines of Assembly code, to then call the syscall. While the chunk of code used for the implant is x64 Assembly often extracted or slightly changed from NTDLL, the intent is just to use that system call in a higher-level language as if it were a normal Native API function.

SysWhispers3. Released in March 2022, SysWhispers3 [15, 16] is a recent fork of SysWhispers2 from just a few months ago, that like SysWhispers2 generates syscall headers and Assembly pairs, allowing them to be used in C/C++ code, as a means to evade hooks. SysWhispers3 includes additional features to help avoid detection. For instance, it will replace the syscall instruction with "W00tW00t"—which it then replaces with the 64-bit *syscall* Assembly instruction. SysWhispers3 also supports generating WoW64 Assembly and header pairs.

Hell's Gate. Released in June 2020, Hell's Gate [9] is a technique to find syscalls dynamically by parsing NTDLL.dll and then locating the syscalls and extracting the SSNs, saving them to memory to be reused. In the white paper for this tool, it describes performing PEB walking to eventually traverse the PE file format, and then it finds the exports table. Once it finds the start of an NTDLL function, it performs what it calls the "pseudo-disassembling," looking for the *mov eax* opcode, 0xb8. Once it finds that, it extracts the SSN by taking the next two bytes, and saving them to memory, where they can be used again. As we have observed in our section 2.5 Changes in Syscall SSNs on Windows 10 and Windows 11, this approach would be problematic since now SSNs can exceed two bytes. Thus, taking just two bytes would sometimes truncate the syscall. This could be improved, however, by simply extracting a DWORD, four bytes. Any value moved into eax must be in the form of a DWORD, so a number that is only two digits would have extra zeros. In order for Hell's Gate to work, it must have an unmodified version of NTDLL. Some EDR products will modify NTDL, such that Hell's gate may not be effective on some desirable NTDLL functions, which can lead to the corresponding syscalls. Dual loading of NTDLL can help allow Hell's Gate to work on a clean version of NTDLL, although there can problems with functions being hooked to load NTDLL.

Halo's Gate. Released in April 2021, Halo's Gate [17] is an extension of Hell's gate that overcomes mitigations from EDR's to prevent Hell's Gate from working. In this implementation, the first several bytes of NTDLL functions are checked to see if they are hooked. If the opcodes for a *jmp* instruction, 0xe9, are present, then it can be programmatically determined that API has been hooked. If so, it will look at the next or previous syscall, and determine if that syscall was not modified by EDR. If not, it then extracts that syscall value. It then will add or subtract based on where that syscall is in relationship to the hooked API. The assumption is that syscalls increment only by one. Thus, this refined version of Hell's Gate can dynamically overcome the hooking. This technique would be foiled by what we observed in manually examining each syscall in the latest builds of Windows 10 and Windows 11. While many do increment only by one, in a predictable fashion, there are often instances where the last two digits may appear that they are incrementing, but the value may have increased from three, to four, to five, or even to six digits. Thus, Halo's Gate would not be effective if there were additional bytes inserted, because the underlying assumption that each syscall is incrementing by one would be false.

FireWalker. FireWalker [18] causes a Vectored Exception Handler to try to locate system call stub after single stepping with use of it of EFLAGS register.

Loading a Copy of NTDLL into Memory. In a 2018 article, Hod Gavriel describes several malware samples that used syscalls, and he noted that year marked a sharp increase in direct syscall usage. Gavriel describes three methods—reading from memory and then two variations of a loading a second NTDLL – one that uses APIs to load part of NTDLL into memory, so that it appears it was loaded normally, and another that simply reads it into memory. Reading from memory involves utilizing NTDLL that was already loaded into memory through normal means.

Dual Load of NTDLL. Dual loading can be done to load NTDLL into memory, such that it appears it was loaded normally. NtMapViewOfSection, NetMapViewOfSeectionEx or NtOpenSection could be called with a new copy of ntdll.dll from disk [1, 6]. While this version of NTDLL could not be hooked, it could be traced via hooking NtMapViewOfSection or NtOpenSection. Once loaded, other approaches could be used to parse the dual loaded version of NTDLL, extracting syscall SSNs,

Dual Loading of NTDLL from Handles. Alternatively, another variation on the above approach is to use NtOpenFile and NtCreateFile, by using a handle to NTDLL to dual load it [1].

Reading NTDLL from Disk. NtReadFile can be used to load it into memory [6]. With this technique, unlike with dual load, it would not appear in loaded memories. This could be detected by hooking NtReadFile.

2.7 Intercepting Syscalls

Microsoft's PatchGuard (PG) prevents changes to important areas of the kernel, meaning some syscalls may not be able to be fully defended against by security products. The only way to intercept syscalls is via user-mode hooks. Thus, hoking System Service Dispatch Table (SSDT) in kernel-mode to intercept syscalls is impossible to hook due to PG. Various other methods are used to bypass AV while invoking syscalls. For instance, Hell's Gate is used to extract the SSN and execute with a code stub, as described elsewhere, although as mentioned EDR can overwrite parts of the relevant NTDLL function, causing Hell's Gate to fail. Those mitigations are user-mode, however, and if creative thinking can allow them to be overcome, then the shellcode author need not fear syscalls being intercepted at the kernel level.

3 ShellWasp

ShellWasp [19] is a tool that creates custom shellcode templates for using syscalls. It conveniently provides a way around the portability issue of shellcode by parsing the PEB and using precomputed lists of syscall values that

correspond to specific releases. ShellWasp is the only tool designed to support the generation of syscalls for shellcode in a 32-bit, WoW64 environment.

3.1 ShellWasp Origins

The inspiration for the method used by ShellWasp to overcome the portability problem of syscalls comes from studying the detailed PEB structure at Vergilius [20], and then finding the values for OSMajorVersion, OS-MinorVresion, and OS BuildNumber, which are located at offsets 0xa4, 0xa8, and 0xac. Having taught the writing shellcode to hundreds of students at an undergraduate and graduate level over several years, I had discussed walking the PEB in great detail via Assembly, with respect to traditional shellcode that utilizes WinAPI functions. Walking the PEB is used to eventually find the base address of a DLL, where the PE file could then be traversed to reach the exports directory. I was aware that both the TIB and PEB can provide a great deal of information through introspection, so my intent was to search through it, to see if I could find anything useful. Better research into the first SysWhispers would have made those efforts redundant, as the first SysWhispers used the PEB in a slightly different way. Instead of using offsets 0xa4, 0xa8, and 0xac from the fs register, which points to the TIB in WoW64, they used offsets 0x118**,** 0x11c, and 0x120, from the gs register, which points to the TIB in 64-bit Windows. When I created this, I was not aware of the original SysWhisper's usage of the PEB for this purpose, as I was more familiar with their more modern techniques. Regardless, we can regard ShellWasp is just an incremental contribution, since SysWhispers had used the PEB first, although in a different fashion.

3.2 Precomputed Syscall Tables

ShellWasp uses syscall tables for Windows. For Windows 7 to Windows 10, it relies upon syscall tables that were prepared by Jurczyk [4] and released online at his web site and GitHub. Those only go up to 20H2, so for recent builds of Windows 10 and Windows 11, we have generated the syscall values ourselves. Originally, this was done using a tool, and then checking results in WinDbg. Due to high errors encountered when using a tool, this is now done in a mostly scripted process with WinDbg[1]. Once results are obtained from WinDbg, they are parsed via a Python script and saved in JSON format. ShellWasp only supports Windows 7, 10, and 11 at the time of writing, as a design choice; others may be supported at a later time. All syscalls tables utilized in ShellWasp are arranged in a new JSON, using values from Jurczyk's previous work as well as new ones we have generated.

3.3 Overview of ShellWasp

Overcoming the Portability Problem in Shellcode. ShellWasp solves the portability problem by using the PEB to identify the OS build. As shellcode is often intended to smaller in size, the user must select the releases to support. For instance, it may be more practical to support only more recent versions of Windows 10 and Windows 11, rather than all possible releases. After all, if a user has automatic Windows updates, then they will have the most current version of Windows, although some may choose to explicitly disable automatic updates. Thus, when using just Windows 10 and 11, ShellWasp does not check the OS major and OS minor versions, as it will only be searching for builds selected by the user. It does check the OS major version if a combination of Windows 7 and Windows 10/11 are used, as each requires a different method to make the syscall. When using just Windows 10 and 11, only the OSBuild is needed, because even if all possible builds are selected, each is unique. ShellWasp will only search for the last byte of the OSBuild, to make the shellcode smaller in size, rather than using a WORD or DWORD as the size, which would unnecessarily increase the size of the shellcode.

[1] This script to parse WinDbg results will be made available on the GitHub.

```
mov ebx, fs:[0x30]
mov ebx, [ebx+0xac]
mov ecx, esp
sub esp, 0x1000
cmp bl, 0xD7            ; 1703 release
jl less1
push 0x1d               ; NtCreateKey
push 0x17e              ; NtSetContextThread
push 0x55               ; NtCreateFile
push 0x8                ; NtWriteFile
jmp saveSyscallArray
less1:
cmp bl, 0xB0            ; Win7, Sp0 release
jl less2
push 0x1a               ; NtCreateKey
push 0x150              ; NtSetContextThread
push 0x52               ; NtCreateFile
push 0x5                ; NtWriteFile
jmp saveSyscallArray
less2:
cmp bl, 0xB1            ; Win7, Sp1 release
jl less3
push 0x1a               ; NtCreateKey
push 0x150              ; NtSetContextThread
push 0x52               ; NtCreateFile
push 0x5                ; NtWriteFile
jmp saveSyscallArray
```

Fig. 11. ShellWasp identifies the OS release. For each syscall selected, the syscall values are pushed onto the stack.

Syscall Array. Once the OS build has been identified, ShellWasp saves the SSNs into a syscall array, so that the SSNs are saved and may be retrieved when needed by the shellcode. In ShellWasp, we call this the syscall initialization. In ShellWasp, the edi register is selected to point to the start of the syscall array. Space for the array is made by moving esp back by 0x1000. Each syscall value is then pushed onto the stack. Once the syscall initialization is completed, the address of esp is moved into edi, which is now used as a point of reference to reach the start of the syscall array. This can be seen in the figure below.

Make space for syscall array.

```
mov ecx, esp
sub esp, 0x1000
```

```
saveSyscallArray:
mov edi, esp
mov esp, ecx
```

Save pointer to sycall array at edi and restore stack.

Fig. 12. The syscall array can be created by making space for it on the stack. Once the syscall values have been pushed onto the stack, edi can be used as a point of reference to access their values.

By using a syscall array, then it becomes much easier to make syscalls, as we do not use values directly, but just reference them from the syscall array. Thus, instead of loading a syscall like *mov eax, 0x18*, ShellWasp would instead do *mov eax, [edi]* or *mov eax, [edi+0x04]*. This allows the syscalls passed to be universal.

```
mov eax, [edi+0xc]      ; NtCreateKey syscall
call ourSyscall
```

Fig. 13. A syscall is made to NtCreateKey by dereferencing an element from the syscall array.

If the shellcode author is combining Windows 7 and Windows 10/11, then there is an extra step added to the initialization sequence, as it checks the OS Major Version as well. That is then pushed on the stack last. It is then

accessible as the dword immediately preceding the start of the syscall array. These extra steps are performed only if needed by the OS releases selected by the user, which is required only when combining Windows 7 with Windows 10/11. Otherwise, ShellWasp creates uses the syscall format supported by the OS.

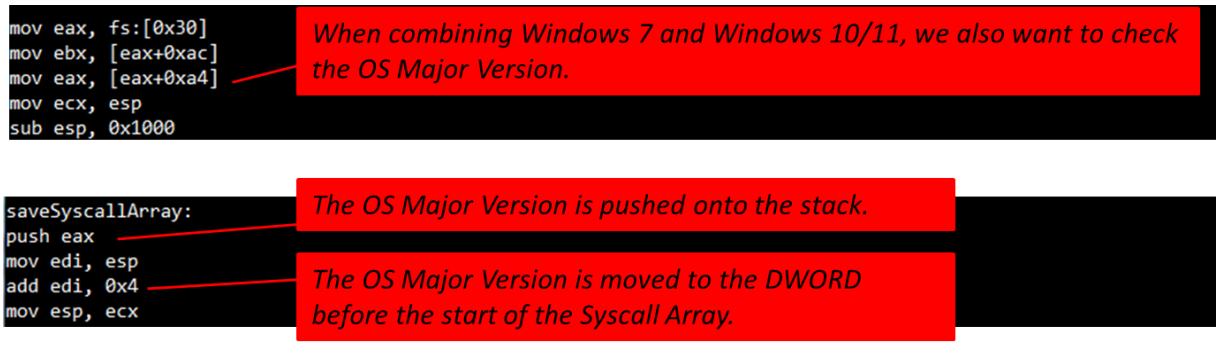

Fig. 14. When combining Windows 7 and Windows 10/11, ShellWasp also checks the OS Major Version. This is saved in the DWORD immediately before the syscall array, for ease of access.

3.4 ShellWasp Syscall Functions for Different OSs.

Windows 7 and Windows 10/11 have very different ways of invoking syscalls in WoW64 shellcode. All of these have been deduced by analyzing NTDLL function stubs. However, reverse engineering can go only so far. Thus, experiments were carried out with each to see if they work in a 32-bit WoW64 shellcode environment, absent anything else. The Windows 7 method is slightly more complex, as it requires more Assembly. The Windows 10/11 method of invoking the syscall is simpler, requiring only two lines of code. Experiments were also carried out to test that one method would not work on the other, without problems. For instance, could a purely Windows 7 method be used on Windows 10, to invoke the syscall? The answer is that it cannot, and so there is a clear distinction between syscalls for each of these.

When designing ShellWasp, the decision was made to make the best shellcode, given the releases that the person was trying to exploit. Thus, if they were only targeting Windows 10, then that would be the method used to invoke syscalls. If it was Windows 7, then it would use the Windows 7 method of invoking syscalls. However, if the shellcode author wanted their code to be more universal and work across Windows 7, 10, and 11, then it is necessary to check the OS Major Versions, as described above, a step which otherwise could be omitted. In ShellWasp, that is saved right before the syscall array, so a shellcode targeting Windows 7 and Windows 10/11 could dereference that address, to see which version of the syscall implementation to use. This is something that might not be knowable when writing the shellcode, and it would depend on the target's OS. In interest of creating more compact and simple shellcode though, these extra steps to check are skipped if the user selects OS releases to target that belong to just one set of OSs. The assumption is the shellcode author may not be targeting everything, but just the ones that are most likely, an interest of making the shellcode size smaller.

Windows 10/11
Syscall Function

```
ourSyscall:
call dword ptr fs:[0xc0]
ret
```

Windows 7
Syscall Function

```
ourSyscall:
xor ecx, ecx
lea edx, [esp+4]
call dword ptr fs:[0xc0]
add esp, 4
ret
```

Windows 10/11 and Windows 7
Syscall Function

```
ourSyscall:
cmp dword ptr [edi-0x4],0xa
jne win7

win10:
call dword ptr fs:[0xc0]
ret

win7:
xor ecx, ecx
lea edx, [esp+4]
call dword ptr fs:[0xc0]
add esp, 4
ret
```

Fig. 15. ShellWasp will select the correct syscall implementation automatically based on the specific OS releases the shell-code author wishes to target.

3.5 Creating a Template for Syscalls

ShellWasp supports all user-mode syscalls, for which we have prototypes, which is nearly all. Users are able to select desired syscalls, in a desired order, and ShellWasp will build the shellcode with syscalls. The output is intended to be a template for users to use as a starting point to customize the parameters. As such, each parameter is given as 0x00000000, and each is labelled with the parameter type and the parameter name. As it is, no syscall will work in the generated, as the values must be provided. The parameters come from an original JSON file that was created laboriously for the SHAREM project, which identifies the number of parameters, the parameter names, and the parameter types. Without knowing the function parameters, there would be no way to generate a template of the syscall shellcode on the fly.

Stack Cleanup and Restoring the Pointer to Syscall Array. Once a syscall is called, the value in edi pointing to the syscall array will be lost. To prevent this, edi is saved by pushing the value in it onto the stack, right before syscall parameters are loaded onto the stack. After each syscall, it is then immediately retrieved. Stack cleanup is done to remove the syscall parameters. At that time, the pointer to the syscall array will be at esp, and thus it can be placed into edi with a simple *pop edi* instruction.

```
push edi                ; Save pointer to sycall array  by pushing it onto the stack.
push 0x00000000         ; PULONG Key
push 0x00000000         ; PLARGE_INTEGER ByteOffset
push 0x00000000         ; ULONG Length
push 0x00000000         ; PVOID Buffer
push 0x00000000         ; PIO_STATUS_BLOCK IoStatusBlock
push 0x00000000         ; PVOID ApcContext
push 0x00000000         ; PIO_APC_ROUTINE ApcRoutine
push 0x00000000         ; HANDLE Event
push 0x00000000         ; HANDLE FileHandle

mov eax, [edi]          ; NtWriteFile syscall
call ourSyscall

add esp, 0x24           ; The shellcode must remove the stack parameters from the stack.
pop edi                 ; Restore the pointer to the syscall array back to edi.
```

Fig. 16. ShellWasp prepares a template for a syscall to NtWriteFile. It has labelled the type and name for each expected syscall parameter.

Syscall Prototypes. ShellWasp uses a custom JSON for syscall prototypes, originally created for the SHAREM project[2]. This was originally created for our shellcode analysis framework, as it supports full emulation of all user-mode syscalls – or at least the 99% that we could find function prototypes for. All the ones from NTDLL undocumented features were taken, and the 30 or so that were on MSDN were also taken. The vast majority of syscalls came from searching for each individually. If the corresponding NTDLL function could be found used in a project with function prototype available, then it was taken. As most of these are undocumented, there is an underlying assumption that the prototypes are more or less correct, and that individuals who created or used them did so in good faith, with a best effort at accuracy. Some may have been reverse engineered and given names based on suspected usage, while others may have come from better documented sources. As numerous sources were used, there was not an attempt to identify the source of each function prototype, and all are used here in good faith to help better support the security community. Each function prototype found was parsed and put into a custom JSON or tuple. For each, there are unique values for name of the syscall, the number of parameters, the parameter types, the parameter values, and whether or not each parameter is IN, OUT, or OPTIONAL. The function prototypes are based on the Hungarian notation used by Microsoft in their official documentation on MSDN.

Config file and Customizing Syscalls. Users may supply desired syscalls in the config file, and they also may be provided as input in the command line interface, by typing one per name. When providing them as input, it is case insensitive. It can be desirable at times to have the same syscall used more than once, and ShellWasp supports doing so. To save space and avoid redundancy, however, it will only initialize each once in the syscall array. Thus, if a syscall is used three times in the shellcode, it will only have one element in the syscall array.

ShellWasp is intended to be highly flexible, so users can easily rearrange the order of syscalls, using the rearrange feature. It only takes seconds to rearrange the order of syscalls. Similarly, specific Windows releases can quickly be added or removed in seconds. ShellWasp allows users to save their current selections in the configuration file. Changes made in ShellWasp can easily be saved in the config file, for future use.

3.6 General Usage of ShellWasp.

Any sequence of syscalls that can implemented in a higher-level language, such as C, C++, or C#, likely could be implemented as shellcode, if done correctly. ShellWasp is intended to simplify that process, firstly by providing an elegant solution to the portability problem. Using precomputed syscall values is the best way to go forward for

[2] SHAREM is intended to be released September 30, 2022. It will be available at https://github.com/Bw3ll/sharem .

shellcode. In new builds of Windows 10 and Windows 11, Halo's Gate likely would not be effective, given that SSNs are no longer incremented by 1, as described elsewhere in this paper. While it could be possible to dual load NTDLL and then do a modified version Hell's gate on it, where it extracts size DWORD instead of size WORD, that could be feasible. However, that likely would involve using the Exports directory and WinAPIs, which we would prefer to avoid, as the code to walk the PEB to find the exports directory is very easily recognizable. The code produced by ShellWasp for the syscall initialization sequence is very small by design, as it does not need to consider ever possible release of Windows. After all, if this is being used for offensive security work, then likely reconnaissance will have already give shellcode authors an idea as to the OS to attack.

To our knowledge, no other project has used the PEB to identify OS builds in a 32-bit, WoW64 environment, although it has already been used in SysWhispers for that purpose, to generate 64-bit Assembly stubs and header files. We are similarly not aware of any shellcode work to use syscalls, outside of Egghunters, except for a 2005, Windows XP era shellcode. That shellcode was made when syscall values changed less frequently, and it had hardcoded the syscall values. We are not aware of any shellcode project that has used a syscall array, as a means to allow the correct syscall values to be easily retrieved at runtime.

As will be described elsewhere, ShellWasp is highly effective at generating the syscall initialization sequence that can be used to start the shellcode, creating syscall arrays, whose elements can be dereferenced at runtime to supply the correct syscall values. In one example, we have a complex shellcode with several syscalls, and which works on both the latest versions of Windows 10 and Windows 11, due to the syscall array.

4 Methodology for Creating Shellcode with Syscalls

Using WinAPIs in shellcode is extremely well documented, and there are numerous works devoted to that topic, and it has been taught in countless courses and numerous security trainings. One of the co-authors of this paper, in fact, has taught creation of modern Windows shellcode to hundreds of students at the undergraduate and graduate level as part of university courses. In other words, there is a very well-established methodology to usage of WinAPIs in modern shellcode. As is well known, this typically involves walking the PEB and then eventually finding a base address of a module, which then enables one to apply the PE file format to that DLL, which then in turn allows us to traverse the PE file, to discover exports. While this can be done one function at a time, clever shellcode will do it in a more modular fashion to maximize code reuse, and often hash values will be used instead of plain text, when checking through the exports table, to maximize stealth. WinAPIs tend to be somewhat straightforward to use in a Windows shellcode context. The same cannot be said to be true for Windows syscalls. At present, prior to this research, how to even use them in a shellcode context would be unclear to the vast majority of users, and even those who are knowledgeable, would need to do some experimentation to resolve some open questions, not to mention figure out a way to address the portability problem with respect to syscalls.

This research helps shed light on these issues and to provide a methodology that can be used by those who wish to build shellcode with syscalls.

4.1 Addressing the Portability Problem with Syscalls in Shellcode

As with any usage of syscalls, the greatest challenge remains the portability problem of syscalls. How can you identify the correct Windows release, so that the correct syscall values that correspond to that release may be obtained? Without being able overcome this hurdle, any shellcode is simply proof of concept, as it would not work in a real-world environment, except on the one release that it was written for.

In our research, we propose a way to address this, which at a high level is somewhat similar to what was done with the first SysWhispers. That is, we must be able to programmatically identify the correct syscall version through Assembly, and then precomputed syscall values can be supplied. This is the approach taken by ShellWasp, and the shellcode it produces is vastly more economical in size than the Assembly stubs for SysWhispers. That is understandable, as these are serving different purposes, and shellcode is more targeted due to what can often be a size limitation with shellcode; SysWhispers need not deal with this restriction. Our recommendation for users is to customize the shellcode for the target, and not to try to make it possible for every possible Windows OS and each release. If the most widely used Windows releases – Windows 10, Windows 11, and Windows 7 – are instead targeted, we could make an assumption that the user most likely receives automatic updates, and in that case, it

might be advisable to just target some of the most recent builds. Because we have a narrower range of targets, it is not necessary to check OS major or OS minor versions, unlike with SysWhispers, as our intent is not to make it universal, but to make the shellcode far more compact and highly likely to be successful on the targets. Additionally, as described elsewhere, when looking at OS builds, in most cases we only need to look at the last byte of the OS Build field from the PEB. The vast majority of OS builds end in different values when truncated to the size of a byte. By comparing a byte rather than a DWORD, our shellcode also is smaller in size.

It is advisable to use Assembly to compare the OS build of the target machine with the supported releases used in the shellcode. For instance, perhaps the shellcode is only looking at the two most recent versions of Windows 10 and Windows 11, and the latest service pack of Windows 7. This can be accomplished with Assembly. As with ShellWasp, likely the best way to utilize the specific syscall values for the desired syscalls, is to simply push them onto the stack. A point of reference can then be used to be able to access the beginning of these, which we can regard as a syscall array. This could be saved in any register, or even on the stack itself, so that it can easily be referred to again. When creating the syscall array, it would be advisable to make room on stack, by subtracting a large value from it, and pushing the values there. First, it is necessary to save the current address of the stack in a register, and then the stack can be restored to its original state, once the syscall array has been created. Once this work has been completed, we now have a pointer to an array whose elements are the syscall values for our specific machine, and the initialization sequence for these takes very little code to implement.

When creating shellcode with syscalls, likely the number of syscalls used will be relatively small. For a simple task, they could be as few as four or five. For more complex functionality, it is feasible there could be as many as 20 or 30 syscalls used, although in that case the size likely would be significantly larger. Such shellcode may be better suited for malware than exploitation, although in some exploits, size is not an issue. Regardless, if multiple syscalls are being used, and if one syscall is being used more than once, then it would be beneficial to have the syscall SSN be initialized for that syscall only once, rather than doing in multiple times. Thus, it can be referred to by the same element in the syscall array, which can be dereferenced to obtain the syscall value.

When writing shellcode with syscalls, as with WinAPIs, it is necessary to push the parameters onto the stack prior to making the syscall. With syscalls, the parameters will remain on the stack, and thus, it is advisable to remove them ourselves. This can be accomplished by adding to esp the corresponding number of bytes taken up by the syscall's parameters, as can be seen in the figure below. Prior to making the syscall, eax should be loaded with the appropriate syscall SSN. Since we are using a syscall array, we can simply dereference it, with an instruction such as *mov eax, [edi]* or *mov eax, [edi+0x4]*. If a syscall array is being used in such a fashion, then care should be taken to ensure that the pointer to it is not overwritten. If using a register to point to the syscall array, then pushing it onto the stack prior to pushing the syscall's arguments and then restoring it by popping it back into the same register, will have the desired effect of ensuring that pointer to the syscall array is not lost.

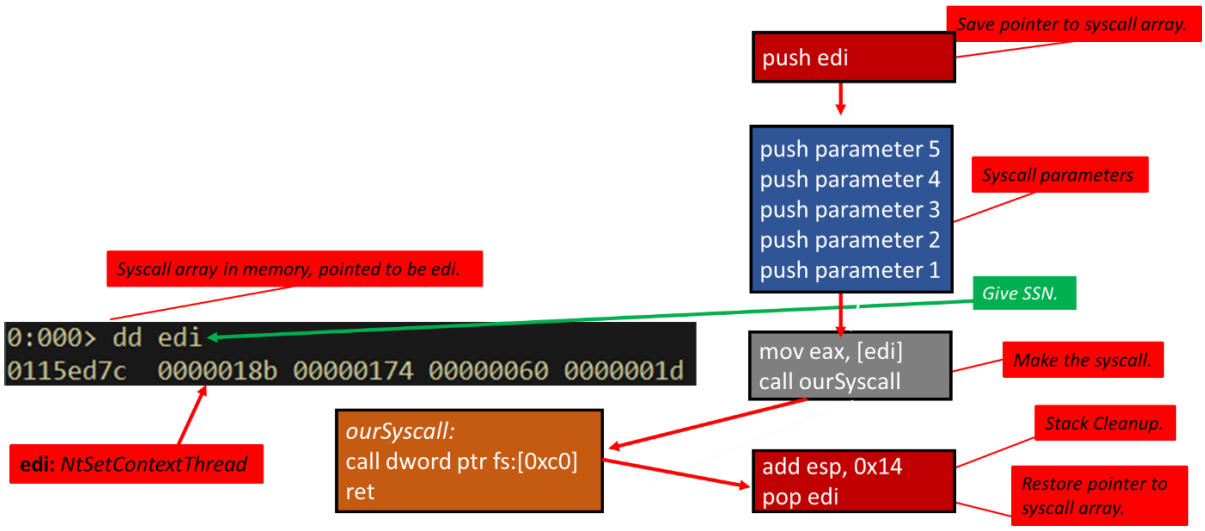

Fig. 17. Syscalls in shellcode can be used by providing the syscall system service number through the syscall array. Here it is pointed to be the edi register.

4.2 Invoking the Syscall in Windows 7 Vis-à-vis Windows 10 and 11

One of the challenges associated with modern syscall shellcode is in getting it to even execute shellcodes correctly in the first place. This is described partly in the section 3.4 ShellWasp Syscall Functions for Different OSs. There are distinct implementations of Windows syscall, for both Windows 7 and Windows 10/11, and they are not compatible, meaning a shellcode written for one, will not work for both, unless the shellcode author is checking for the OS Major Version and selected the proper implementation of the syscall. Additionally, these should be placed within a function. One general function could be used, and we could call it something like ourSyscall. As described in the corresponding ShellWasp section on this topic, a shellcode can easily be compatible for both variants – the Windows 7 and Windows 10/11 WoW64 implementations. In that case, it must programmatically be determined what the correct OS is, so that the right syscall implementation can be used. In ShellWasp, that address is determined earlier in the shellcode, stored right before the syscall array. Shellcode authors could do that, or they could dynamically determine the current OS by walking the PEB and looking at the OS Major Version inside the syscall function.

4.3 Complexity of Setting Up Syscalls in Shellcode

Syscalls can be considerably more difficult to use than Windows APIs, depending on the desired tasks at hand. In many cases, it may be necessary to generate handles to specific resources. Some steps require considerably more setup than would be done with corresponding Windows APIs; in fact, some may require several syscalls, to achieve what could be done more easily with WinAPIs.

Shellcode authors may wish to choose to focus on functions from kernel32.dll, as those have corresponding NTDLL functions. However, there are less than 600 user-mode syscalls. (That is, if we regard syscalls that begin with Zt and Nt as being one in the same.) That means that a lot of functionality that is available via kernel32.dll may not be as easily available via syscalls. In some cases, for a particular WinAPI, it might be possible to replicate its functionality via syscalls. However, those could involve several syscalls and a great deal of setup; for others, there is a more one-to-one correspondence, and minimal setup may be required. In many cases, when using syscalls in shellcode, the amount of work necessary may be impractical to do something from a purely syscall standpoint, due to the amount of setup that would be required. If not already known to the shellcode author, the only way to identify the syscalls that correspond to different API's is to trace them through a disassembler or a debugger. It is also important to note that not all NDLL functions have a corresponding syscall; in fact, there are many NTDLL functions that will not directly make any syscalls, but indirectly they call other functions, and they may make multiple syscalls.

When attempting to build shellcode with syscalls, if there is a known way to achieve certain functionality using APIs, then a first step would be to see what, if any, syscalls they may directly correspond to. Next, the user must identify the parameters required for each. It is very common for NTDLL functions to have additional parameters, and many require handles, as opposed to their kernel32 counterparts. Some parameters may be structures that need to be implemented, although in many cases some values may be optional.

In some cases, although it may be possible to identify NTDLL functions with corresponding syscalls that appear to mirror kernel32 functions, the setup and requirements may be very complex and challenging, requiring a great deal of time and effort. When trying to implement these in shellcode, it is best to look to see how these may be used normally in programs. Looking at them in a debugger often is best, as it allows one to better understand how they operate in a more dynamic fashion, although a disassembler can be useful as well. When analyzing potential syscalls or APIs in a debugger, it is important to note that while for some there may be elaborate setup, not all of that may be required for the desired functionality. That is, some parameters may be optional and could be skipped when attempting to reimplement it as shellcode, thereby reducing the labor required as well as the size of the shellcode.

Syscalls in Shellcode Vis-à-vis Shellcode in Red-Team Software. All the syscall tools surveyed in the literature review are designed to facilitate using syscalls in programs written in high-level languages, even if some may use Assembly, such as for creating syscall Assembly stubs. The vast majority of the software is written in a high-level language, with only a very small amount being in Assembly. Because syscalls are typically being used in high-level languages as a substitute for an NTDLL function call, it is easier to set up and use syscalls in high-level

languages, as necessary handles or other resources can be easily provided through what is provided by the high-level language. When trying to do equivalent actions in shellcode, many otherwise simple tasks may require a great deal of set up. Additionally, there is the issue with shellcode being position-independent. Shellcode is not aware of where it is in process memory, and it must utilize GetPC gadgets, such as *call exx* / *pop exx* instructions, to even establish where it is in memory. Any Windows APIs used in shellcode must be set up by using PEB walking and then traversing the PE file format, to find the exports directory, etc.

4.4 Understanding Syscalls Output

There is a marked difference between syscalls and WinAPIs in terms of output. More specifically, we could say that this is true of NTDLL functions vis-à-vis WinAPI functions. With respect to the WinAPIs, often an important value will be returned in the eax register. Sometimes it is the most important or only output value associated with the Windows API. For instance, with LoadLibrary, a handle to the library is returned in eax; that is true whether it is already loaded or newly loaded, and the handle is identical to the base address of the library. With syscalls though, all or virtually all return the same value in eax – the NTSTATUS. If the syscall is successful, then it will return 0x00000000 into eax, STATUS_SUCCESS. There are dozens of other hex values that may be returned into eax as message identifiers, indicating, for instance, if something went wrong with the syscall. With NTDLL functions, including corresponding syscalls, output values of interest will be returned as output parameters, as can be seen in the figure below. Additionally, NTDLL functions may also use entire structures as output parameters.

C++ Copy

```
__kernel_entry NTSYSCALLAPI NTSTATUS NtAllocateVirtualMemory(
  [in]      HANDLE    ProcessHandle,
  [in, out] PVOID     *BaseAddress,
  [in]      ULONG_PTR ZeroBits,
  [in, out] PSIZE_T   RegionSize,
  [in]      ULONG     AllocationType,
  [in]      ULONG     Protect
);
```

Fig. 18. Hungarian notation for NtAllocateVirtualMemory from MSDN. Two parameters are used for output.

Because NTDLL functions – and, thus, all syscalls – use output parameters to return the most important data, then shellcode authors must be cognizant of this as well. Although this may appear more confusing at first to newcomers, it is relatively simple to use. The shellcode author can provide any value in memory to be an output pointer, so long as it has appropriate RW permissions. Frequently, sophisticated shellcode will check and reuse output values in subsequent functions – be it WinAPI calls or syscalls. This requires the shellcode author to be aware of where in memory output may appear.

5 Evaluation

With original scientific contributions, it is necessary to provide some validation that shows that the research works well and is effective; in this section, we provide evidence that our research has been validated. Firstly, with ShellWasp, the tool, validation is achieved by showing the accuracy and correctness of the shellcode and Assembly it creates. Secondly, the case studies below provide validation, showing how ShellWasp can be used to create actual shellcode that uses multiple syscalls to achieve complex, malicious functionality. The tool itself additionally is an embodiment of a novel technique created by this research to overcome the portability problem with syscall shellcode, and that too can be used outside of the context of the tool; the case studies stand as evidence in support of this. Finally, this research has proposed a new methodology for creating shellcode with syscalls. Prior to this

research, there was no existing methodology for creating modern syscall shellcode, and even basic usage questions for modern Windows OSs had not been addressed. These case studies below provide support and validation for the efficacy of this methodology.

5.1 Validation for the Correctness and Accuracy of the Shellcode Produced by ShellWasp

Validation for ShellWasp can be provided by demonstrating its efficacy. Firstly, the accuracy and correctness of the shellcode produced by ShellWasp illustrates its efficacy. Secondly, the correctness and accuracy of the shellcode produced by the tool has been by virtue of the fact it can be adapted and used to create real-world, syscall shellcode, and that the desired shellcode works in multiple OSs and releases without problems.

To establish the accuracy of the shellcode produced by ShellWasp, reverse engineering tools like WinDbg have been used to carefully inspect the quality and correctness of the generated Assembly, while ensuring that it has the desired effect, instruction by instruction. For instance, numerous tests were performed to demonstrate that the correct release version was identified, and that the syscall values for the selected syscalls were pushed onto the stack, and that they resulted in a syscall array, whose location could be saved in edi and used subsequently in the exploit. This was done through stepping through the code in the debugger, with the output produced by ShellWasp, to ensure it had the desired effect with every Assembly instruction.

Next, it was demonstrated that even though the syscall itself would modify the value in edi, that after each syscall and the corresponding stack cleanup, that the pointer to edi was resolved correctly. Additionally, it was verified that the syscalls used in ShellWasp would appear in the shellcode in the correct order, and that the correct syscall value for each would be in the correct element, based on the Windows release. Dozens of experiments were performed to ensure that even if the same syscall was used multiple times, that it would only be initialized once in the syscall array, and that that element would be used each time it was needed, rather than repeatedly re-initializing the same syscall value as different elements. By closely analyzing the code produced by ShellWasp visually – as well as by doing so with the use of a debugger – we have provided assurance that the code produced has the desired effect. Dozens of experiments were carried out with many variations of different types of inputs for syscalls, to ensure that the accuracy and correctness of the Assembly in the shellcode would not be compromised. For instance, there were examples with just one or a few syscalls used, as well as ones with dozens used, and with the same ones being used several times. Validation for the Assembly in the shellcode was also provided by the usage of Keystone-engine, which compiles the Assembly instructions into bytes; any errors in Assembly would result in it failing to compile, generating and error message.

One of the features of ShellWasp is its ability create templates for nearly all user-mode syscalls. It does so by utilizing an original, custom JSON/Python dictionary that we created, and as long as the values in the JSON are accurate, then so too is the output produced by ShellWasp, as it is simply a matter of enumerating the values for selected syscalls. Regardless, these were also inspected to see if there were any errors present.

To demonstrate the efficacy of ShellWasp, it must also be established that that the Assembly produced by it works when adapted and used to create real-world shellcode with multiple syscalls used to achieve malicious functionality. It must be able to identify the correct OS release, allowing the shellcode to work on multiple operating systems and different releases, without problems. This was done by upgrading an older Windows XP era shellcode from 2005. That shellcode, from a bygone era, utilizes an entirely different method of calling syscalls, as it uses the int 0x2e instruction. This upgrade was done by a VERONA Labs member, Shelby VandenHoek, rewriting and upgrading the shellcode from scratch for Windows 7 SP 1, Windows 10 21H2, Windows 11 21H2. ShellWasp was used to provide a template to follow and adapt from, with respect to identifying the OS release, initializing the syscall array, and utilizing the syscall array properly throughout the shellcode. At no point was a syscall SSN provided to eax as hardcoded value; instead, each was dereferenced and came from the syscall array. This shellcode was indeed successful on both the most recent versions of Windows 7, Windows 10, and Windows 11. This shellcode is described more fully in the case study, later in this section.

5.2 Case Study for Shellcode with Syscalls: Launching a Secondary Shellcode in a Remote Process

The shellcode in this case study uses several syscalls to provide advanced functionality. This syscall is able to search through active processes and inject a secondary shellcode – in this case, a Meterpreter payload – into

another process. This shellcode utilizes the following six syscalls to complete this work: NtQuerySystemInformation, NtOpenProcess, NtAllocateVirtualMemory, NtWriteVirtualMemory, NtCreateThreadEx, and NtWaitForSingleObject.

NtQuerySystemInformation is used to loop through all active processes. One of its output parameters is an array of SystemProcessInformation structures. Each entry contains a wealth of information, including a pointer to the next process structure, a process's unique process ID (PID), and the name of the process in Unicode string. The pointer to the next process in the list could be obtained through the first element of the structure which is NextEntryOffset.

NtOpenProcess is used to open the desired process with the PID it was able to obtain. As part of this, one input parameter is the OBJECT_ATTRIBUTES structure, which should be provided. The PCLIENT_ID additionally should be provided using the PID obtained in a previous step using NtQuerySystemInformation.

NtAllocateVirtualMemory then is used to allocate memory in the targeted process. This is the lowest layer of the of the well-known api VirtualAlloc, or VirtualAllocEx.

NtWriteVirtualMemory is used to write the contents of the secondary shellcode into the remote process. The secondary shellcode is a separate Meterpreter shellcode.

NtCreateThreadEx is used to spawn a new thread in the remote process. NtWaitForSingleObject is used to wait for the thread to finish execution.

The shellcode in this case study was originally made just to target one specific Windows OS. Later, after completing the shellcode, ShellWasp helped provide ideas for how to implement the syscall array. This permitted the shellcode to be effective across multiple Windows releases, with the exact syscall SSN values being determined at runtime, and using values prepopulated in the syscall array. In the final version of this case study shellcode, at no point is a hardcoded SSN used to load a syscall.

5.3 Case Study for Shellcode with Syscalls: Re-Imagining Windows XP Shellcode for Windows 10 and 11

Outside of Egghunters, an exhaustive search revealed no significant shellcode with more than one non-Egghunter syscall. The one and only exception was a shellcode [5] by Piotr Bania from 2005. At that time, there was no practical solution to the portability problem with shellcode, and syscall values remained identical across service packs, so it was not nearly as problematic as it is today. Thus, hardcoding syscall values, while limiting, was not as problematic as today. The 2005 syscall shellcode also used a dated method of invoking syscalls, *int 0x2e*, which was deprecated since Windows 8, although it has been incorporated into newer, 64-bit implementations of syscalls on Windows 10, albeit behind the scenes.

The 2005 shellcode was also a reflection of the era in which it was written, and it was built to compiled by MASM, with macros. The re-imagined shellcode uses the same syscalls, but it has been completely rewritten by a VERONA Labs researcher, Shelby VandenHoek. Aside from using the same syscalls, there is little to no similarity between the two shellcodes.

The shellcode for this case study is intended to cause a binary to be executed. Like Bania's shellcode [5], this does it in a roundabout way. As Bania explains, the equivalent actions for the WinAPI CreateProcess as a syscall would require several steps to be completed—none of which would need to be considered if just simply using the WinAPI. Firstly, the executable would need to be opened, and then a Windows executive object would need to be created. Next, a thread would need to be created for it, and then the Win32 subsystem would need to be made aware of this. Execution would then need to commence with the thread of the new process. The address space of the new process, from our new binary, would need to be initialized, which includes loading necessary DLLs [5]. All of that functionality cannot be emulated via NtCreateProcess, and with changes in Window 10, likely NtCreateUserProcess would need to be used instead. Being able to effectively use NtCreateProcess was extremely complex in Windows XP, and it is only more so in Windows 10 and 11. As an alternative to using a syscall equivalent to NtCreateProcess, the shellcode in our case study simply creates and initializes a registry key, causing the binary – in this case calc.exe – to execute once the computer reboots. This method has the added benefit of establishing persistence.

The shellcode for this case study creates a registry key with NtCreateKey. That syscall requires an Object Attributes structure to be pushed, which must be initialized correctly. In this example, the registry path is marked as being case insensitive. It also uses a PUNICODE_STRING structure with the supplied path of

"\Registry\Machine\Software\Microsoft\Windows\CurrentVersion\Run\0". It also specifies an access path of KEY_ALL_ACCESS | KEY_WOW64_64KEY, meaning it will use the normal registry, and it will be populated at "Computer\HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run". The shellcode provides other additional access masks, which are commented out, but provides flexibility. Finally, a handle is provided for the registry key. Thus, in order for this syscall to work, multiple parameters needed to be initialized with appropriate values, including pointers to two structures, PUNICODE_STRING and POBJECT_ATTRIBUTES, each populated with correct values, even if some are just null values.

```
; _OBJECT_ATTRIBUTES
    xor edx, edx
    xor ecx, ecx
    push edx ; SecurityQualityOfService = NULL
    push edx ; SecurityDescriptor = NULL
    inc ecx
    shl ecx, 6
    push ecx ; Attributes = OBJ_CASE_INSENSITIVE = 0x40
    push dword [ebp-20] ; US_REG_PATH
    push edx ; Root Directory = NULL
    push 0x18 ; Length
    mov [ebp-24], esp ; OBJECT_ATTR
```

Fig. 19. In the re-imagined syscall shellcode, we can see the POBJECT_ATTRIBUTES structure is initialized with values for use in NtCreateKey. A pointer to this structure is then saved at ebp-24.

The next syscall utilized is NtSetKeyValue. The handle that was created for NtCreateKey must be provided to this syscall in addition to the path to our binary, which in this case is calc.exe. A PUNICODE_STRUCTURE is provided for the ValueName, "Syscall Created Key".

NtClose is then used to close the handle to the registry. The only needed parameter for this is the handle itself. The final syscall for this shellcode is NtTerminateProcess. This kills the process that the shellcode was executed inside.

In this case study, the syscall initialization is utilized, as described elsewhere. The PEB is used to capture the OS Build of the operating system, and the syscall NtCreateKey, NtSetValueKey, NtClose, and NtTerminateProcess are initialized, each having one entry in the syscall array. This particular shellcode supported the latest release for Windows 11, 21H2, as well as the latest release for Windows 10, 21H2. Several other recent releases from Windows 10 were supported, such as 21H1, 20H2, 2004, 1909, and 1903. The most recent version of Windows 7 was also supported. All though this does not include every existing release, it does have the most recent versions of the supported operating systems, so most likely it would work, and additional OSs could be added if desired. This shellcode was tested on Windows 7 SP1m Windows 10 21H2, Windows 11 21H2, and it worked correctly for each. The correct SSNs are initialized in the syscall array, allowing for the correct syscall to be provided at runtime.

```
    mov eax, [edi+4]
    call ourSyscall
    add esp, 24
    pop edi ; Get syscall array
```

Fig. 20. The case study shellcode utilizes the syscall array, retrieving the syscall system service number at runtime. The pointer to the syscall array is restored after the call to the syscall ends.

Because the shellcode in this case study must support Windows 7 as well as newer operating systems, it was necessary to have a dual purpose syscall function. In this case, the OS Major Version is not saved directly in front of the syscall array, as done by ShellWasp. Instead, the shellcode checks it each time the syscall is made, allowing the correct implementation to be utilized. This can be seen in the figure shown below.

```
syscallFunc:
    mov ebx, fs:[0x30]
    mov ebx, [ebx+0xa4] ; OS Major Version
    cmp bl, 10
    jne win7
    win10:
        call [fs:0xc0]
        ret
    win7:
        xor ecx, ecx
        lea edx, [esp+4]
        call [fs:0xc0]
        add esp, 4
        ret
```

Fig. 21. The case study uses a syscall function that works for both Windows 7 and Windows 10, detecting the OS at runtime.

6 Conclusion

Until very recently, syscalls in shellcode simply were not done, outside of some Egghunters, intending to find a tag belonging to another shellcode by searching through process memory. With this research, we provide a definitive way to utilize syscalls in shellcode, overcoming the portability problem. When the first known non-Egghunter shellcode was created 17 years ago [5], the need for supporting different releases was nowhere the problem that it is today, where there are more than a dozen unique releases for just Windows 10. Although the common logic dictates that syscall values frequently change, that often happens not to be true for some frequently used syscalls, which may persist for several releases across an OS, as was the case with one of our case studies, where the syscall SSNs were identical for all versions of Windows 10 and 11. Yet, clearly, that will not always be the case, and to blindly and haphazardly assume such would lead to shellcode that would likely fail. The approach taken by this research has been to embrace using precomputed syscall values, tailored to not every single Windows release in existence, but just the ones that the shellcode authors deem most likely to be used for a specific target, allowing the shellcode to achieve a vastly smaller size, than would otherwise need to be the case.

Utilizing the PEB to identify the OS Build in ShellWasp naively was done through happenstance and looking closely at the members of the PEB structure, as described elsewhere here. Regardless, the PEB is how the first SysWhispers identified specific OS builds, though with 64-bit rather than 32-bit code. Once criticism leveled at SysWhispers was that it was too reliant on the syscall tables [4] created by another individual. While ShellWasp actually uses some syscall tables from the same source, we also are not reliant on it, in that we have created our own for new releases and plan to do so for all future OS builds that we decide to support. Moreover, we provide an alternative way to generate these – one for which there can be no errors. Thus, should we decide to abandon the project, others can continue it.

This research has also looked closely at the changing nature of the syscall values themselves, as their predictability has changed in a way that would make some techniques likely not work all the time, such as Halo's Gate. As described elsewhere, this is due to the fact that, while the last byte of the syscall values tend to increment by one, it may suddenly become instead of a two- or three-digit value, a four-, five- or six-digit value. For instance, we present this hypothetical example: Syscalls incrementing from 0xBA, 0xBB, 0xBC, to 0x123BD is not always predictable, as we would have no way to anticipate 0x123BD. As we had not seen such in earlier syscall tables published online [4], we assume this likely is a defensive mitigation by Microsoft. And while there are many very elaborate techniques to dynamically find syscall values at runtime, there can be other issues. Hell's Gate, for instance, can be disrupted by EDR, that overwrites important bytes in the NTDLL functions for the desired syscalls, making some of them not able to be discovered via Hell's Gate. And, even if parts are not overwritten by EDR, as not all are, then the fact that some syscall SSNs are larger than a WORD would cause some to be incorrectly missed—likely part of what we suspect is a mitigation effort. A simple correction for that could be to extract a DWORD, rather than the WORD described in the original Halo's Gate white paper [9].

Additionally, Halo's Gate, ideally could be an effective way around parts of NTDLL functions being overwritten by EDR, but as mentioned, this technique will not always work if the underlying assumption that each syscall is incremented by one is simply not true, which is now the case. While it is not true for many syscalls, currently

most do increment by one, so the technique would have some successes and failures. Additionally, while we can do other techniques, such as dual loading NTDLL and then applying some variation of Hell's Gate on the secondary NTDLL, some of the functions used to load NTDLL potentially can be hooked. Furthermore, even if that is not so, dual loading NTDLL and then parsing it with Hell's gate would be extra bytes that should be avoided to minimize shellcode size. In much more elaborate red team software or malware that may use syscall shellcode – rather than exploitation being the delivery mechanism for the shellcode—there are not size restrictions to be concerned with. With custom red team software, more elaborate techniques could be embraced with syscall shellcode, assuming they address any underlying mitigations, as alluded to earlier. In short, we argue that using precomputed syscall values is the best and only realistic choice for compact shellcode.

This research has made a very minor, incremental contribution, by providing an alternative way to determine syscall values. Some existing tools look at byte patterns and then extract values, but some can be problematic, if underlying assumptions are not correct. Our solution is to generate a series of simple commands that can be copied and pasted into WinDbg on an OS Build lacking EDR. This results in one line of accurate disassembly for each syscall, showing it moving the syscall system service numbers into eax, rather than an approximation by looking at byte patterns. A simple Python script can then parse the output, saving the correct syscall system service number with the correct syscall in a Python dictionary / JSON. In all, this entire process, while involving a very small amount of manual labor, could be completed in just a few minutes. This is not to imply that every tool that parses NTDLL to extract SSNs is faulty, as we have not in any way attempted to survey all of them. We just encourage shellcode authors or other users of syscalls to be careful and verify accuracy of SSNs.

ShellWasp provides an easy and quick way to quickly create a template for syscall, while addressing the portability problem. This tool provides an excellent starting point for those wishing to create their own syscall shellcode.

The methodology for creating traditional modern Windows shellcode is very well established, with PEB walking and using the exports table to identify and then use WinAPIs. The methodology for doing so with syscall shellcode did not exist prior to this research. While this does not make creating syscall shellcode any easier, it does make it viable, and it provides a starting point for shellcode authors who want to take their work to the next level.

6.1 Contributions

This research has made significant contributions by providing a methodology for how to do create syscall shellcode, something that simply had not been done before. While there are many tools that deal with syscalls, this is all done in 64-bit mode. Virtually all Windows shellcode is 32-bit, and preexisting ssycall tools use the syscalls in a decidedly non-shellcode fashion. In other words, there are no other research remotely similar in scope ours. Not only have we overcome the portability problem with respect to syscall shellcode in 32-bit, WoW64 Windows, but we have even provided a methodology for how to use syscalls in shellcode. This research made a contribution by coming up with the model of a syscall array, a simple yet elegant way to save and use syscall SSNs in shellcode, without the need to use hardcoded values, allowing for the right syscall values to be provided at runtime. A practical contribution has also been made with respect to the tool ShellWasp, which automates much of the work to be done in creating syscall shellcode, by creating a complete template of syscall shellcode, along with a syscall initialization sequence and usage of a syscall array, to achieve portability across different Windows releases.

6.2 Future work

One argument we anticipate is that this work is tied to 32-bit shellcode, not 64-bit. However, having created a new, currently unreleased shellcode analysis framework, SHAREM, which supports emulation of more than 12,000 WinAPIs and nearly all user-mode syscalls, for both 32-bit and 64-bit shellcode, we have searched far and wide for 64-bit Windows shellcode, and we can safely say that there just are not that many out there in the public domain, relative to 32-bit shellcode. Only a very small fraction of Windows shellcode is currently 64-bit. Thus, we can say that largely the world is not writing 64-bit shellcode, although they certainly could. Much exploitation in user-mode occurs in a 32-bit environment, which requires 32-bit shellcode, and most malware tends to be 32-

bit (or mostly 32-bit, if it invokes techniques like Heaven's gate), to allow for it to reach more targets more easily. In fact, some shellcode from malware that we have seen can be both 32-bit and 64-bit, having different entry points for each, although that is not common. Syscalls in 32-bit, WoW64 shellcode certainly would not work in 64-bit shellcode, at least using the methods we have described in this paper. As we have not in any way investigated the issue, we cannot speak to the feasibility of 64-bit shellcode with syscalls. We would hope that it would be viable and simple to utilize syscalls, though in a different way, but without conducting experiments of our own, we are loathe to make any comments on its feasibility.

It is our strong desire that more people will start creating and using more shellcode with syscalls. What malicious or even useful, non-malicious functionality can be implemented via syscalls? Certainly, doing so would require open-minded creativity and frequent use of a debugger to achieve success. And, if doing this as simply a creative activity, it can be useful to remember that size restrictions are not always an issue with shellcode. Shellcode is also used by malware, and in that case, the shellcode typically would not need to be constrained by file size. In fact, recent software [21] allows PE files to be converted to shellcode and used within malware, and the shellcode that results from this can be orders of magnitude larger than handwritten shellcode, which is more the focus of this research.

For us the most highly desirable shellcode invokes pragmatic, highly complex malicious functionality in pure syscalls – or as close to it as can be feasible, minimizing WinAPI usage.

Part of this research has been supported by NSA Grant H98230-20-1-0326. Bramwell Brizendine and Shelby VandenHoek have also been supported by this grant.

References

1. Research, Mds.: Bypassing User-Mode Hooks and Direct Invocation of System Calls for Red Teams, https://www.mdsec.co.uk/2020/12/bypassing-user-mode-hooks-and-direct-invocation-of-system-calls-for-red-teams/
2. Van Eeckhoutte, P.: Windows 10 Egghunter (Wow64) and More, https://www.corelan.be/index.php/2019/04/23/windows-10-egghunter/
3. Stevens, D.: Hancitor Maldoc Bypasses Application Whitelisting, https://isc.sans.edu/forums/diary/Hancitor+Maldoc+Bypasses+Application+Whitelisting/21683/
4. Jurczyk, M.: Windows X86-64 System Call Table (XP/2003/Vista/2008/7/2012/8/10), https://j00ru.vexillium.org/syscalls/nt/64/
5. Bania, P.: Windows Syscall Shellcode, http://piotrbania.com/all/articles/windows_syscall_shellcode.pdf
6. Gavriel, H.: Malware Mitigation when Direct System Calls are Used, https://web.archive.org/web/20220204174358/https://www.cyberbit.com/blog/endpoint-security/malware-mitigation-when-direct-system-calls-are-used/
7. Margaritelli, S.: On Windows Syscall Mechanism and Syscall Numbers Extraction Methods, https://www.evilsocket.net/2014/02/11/On-Windows-syscall-mechanism-and-syscall-numbers-extraction-methods/
8. Hutchins, M.: Windows 10 System Call Stub Changes, https://www.malwaretech.com/2015/07/windows-10-system-call-stub-changes.html
9. Laîné, P., Smelly__vx: Hell's Gate. (2020)
10. de Plaa, C., Stanhegt: Dumpert, https://github.com/outflanknl/Dumpert
11. de Plaa, C.: Red Team Tactics: Combining Direct System Calls and sRDI to bypass AV/EDR, https://outflank.nl/blog/2019/06/19/red-team-tactics-combining-direct-system-calls-and-srdi-to-bypass-av-edr/
12. ElephantSe4l: FreshyCalls: Syscalls Freshly Squeezed!, https://github.com/crummie5/FreshyCalls
13. ElephantSe4l: FreshyCalls: Syscalls Freshly Squeezed!, https://www.crummie5.club/freshycalls/
14. T., J.: SysWhispers2, https://github.com/jthuraisamy/SysWhispers2
15. KlezVirus: https://github.com/klezVirus/SysWhispers3, https://github.com/klezVirus/SysWhispers3
16. KlezVirus: SysWhispers is dead, long live SysWhispers!, https://klezvirus.github.io/RedTeaming/AV_Evasion/NoSysWhisper/
17. Reenz0h: Halo's Gate - twin sister of Hell's Gate, https://blog.sektor7.net/#!res/2021/halosgate.md
18. Winter-Smith, P.: FireWalker: A New Approach to Generically Bypass User-Space EDR Hooking,

https://www.mdsec.co.uk/2020/08/firewalker-a-new-approach-to-generically-bypass-user-space-edr-hooking/

19. Brizendine, B.: ShellWasp, https://github.com/Bw3ll/ShellWasp
20. Storchak, S., Podobry, S.: 1903 19H1 (May 2019 Update) _PEB, https://www.vergiliusproject.com/kernels/x86/Windows 10/1903 19H1 (May 2019 Update)/_PEB
21. Doniec, A.: PE to Shellcode, https://github.com/hasherezade/pe_to_shellcode